Paste a long link, get a short one

```
https://example.com/blog/2026/10/how-a-viral-link-took-down-our-server?utm_source=newsletter&utm_campaign=oct
```



https://short.example/FXsk



click

301 → the long page, in under 100 ms

```
HTTP/1.1 301 Moved Permanently
Location: https://example.com/blog/2026/10/how-a-viral-link…
```

the browser follows it on its own

What short links are for

fit a link where space is short

a post, a text message, a QR code: short.example/FXsk

pick a readable name

short.example/launch-day

count who clicked

launch-day: clicks per day

take a bad link down

410 Gone: this link was removed

How far does one file on a €5 server get you?

And what does it take to handle almost a billion clicks a month?

Everything that will happen, locked before we build

E01	Requirement	Launch: friends and family, 1 click/s
E02	Load	1% of Bitly: 100 clicks/s, 10 million links
E03	Requirement	Custom names: 50 people want the same name at once
E04	Load	A link goes viral: 2,000 clicks/s on one link
E05	Requirement	Click counts per link, within 1%
E06	Failure	Power cut, without warning
E07	Failure	The machine and its disk are deleted
E08	Load	10% of Bitly: 1,000 clicks/s, 100 million links
E09	Requirement	Takedowns: removed links stop within 60 s

Always true, in every event

does	paste a link → a code of at most 7 letters and digits open a code → 301 to the link · unknown code → not found HTTPS only
never	lose a confirmed link, or send it to the wrong page give a code two owners
fast	`redirect p99 < 100 ms · create p99 < 300 ms · errors < 0.1%`

---

why those times · Jakob Nielsen, usability researcher (1993):

`0.1 s` feels instant — **`1 s`** keeps your train of thought — **`10 s`** you lose attention

p99, not the average: at 1,000 clicks/s the slowest 1% is still 10 people every second

Where the traffic numbers come from

Bitly, per year: **100 billion clicks · 2 billion new links**

bitly.com/pages/about, checked 1 Oct 2026

	per year	÷ 31.6 M s = average	× 3 = peak	we test
1% of Bitly	1 B clicks	32/s	95/s	**100/s**
	20 M links	0.63/s	1.9/s	**2/s**
10% of Bitly	10 B clicks	317/s	951/s	**1,000/s**
	200 M links	6.3/s	19/s	**20/s**

links stored = six months of new links: 10 M at 1% · 100 M at 10%

back to a month: 333/s on average × 2,629,800 s ≈ 877 million clicks

333/s = the 1,000/s we tested ÷ 3 · exactly 10% of Bitly: 317/s × 2,629,800 s ≈ 833 million

31.6 M s = a year · 2,629,800 s = an average month · × 3 = the busiest time of day (traffic model)

E01

Requirement

Launch

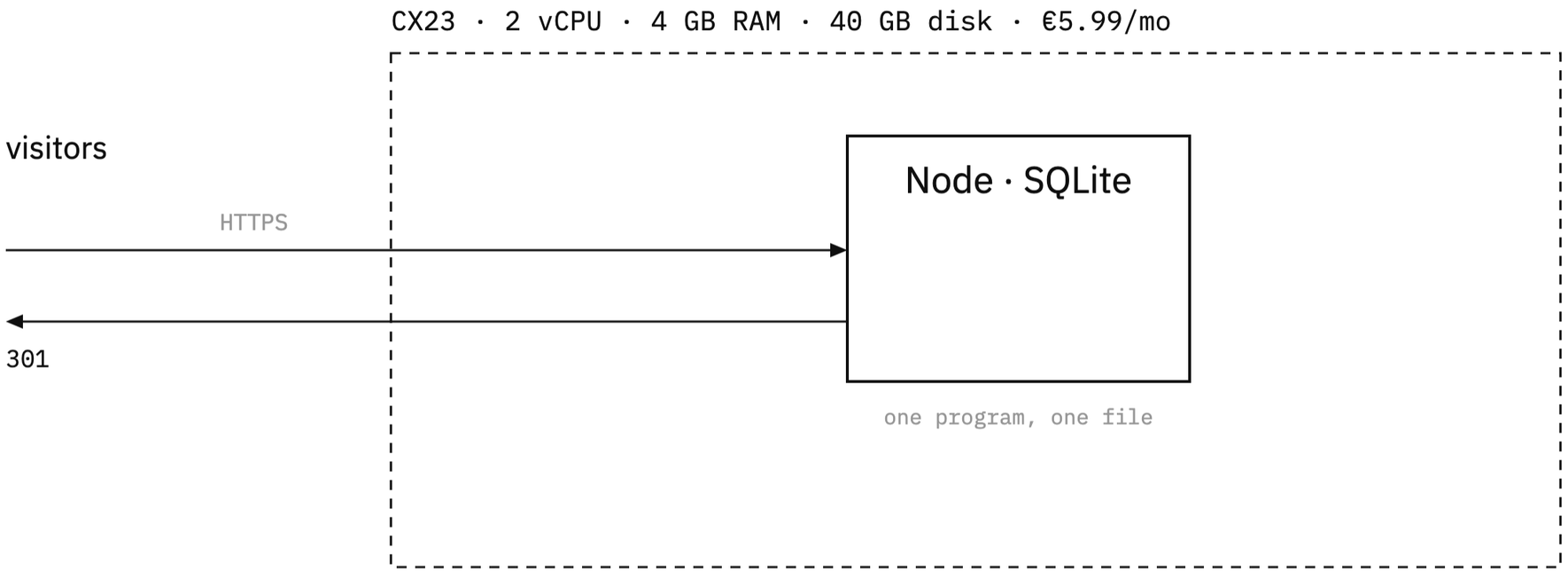
1 click a second and a new link about once a minute, with 1,000 links stored.

why it matters

Day one of any product. Does the simplest possible thing even work?

pass: `the always-true rules`

v0


CX23 · 2 vCPU · 4 GB RAM · 40 GB disk · €5.99/mo
visitors
HTTPS
301
Node · SQLite
one program, one file

v0: one table

links

id	url	created_at
10000000	https://news.example.invalid/…/story-10000000	1791123320
10000001	https://news.example.invalid/…/story-10000001	1791123320

divide by 62 until nothing is left; each remainder is one character:

```
10,000,000 ÷ 62 = 161,290  r 20 → k
   161,290 ÷ 62 =   2,601  r 28 → s
     2,601 ÷ 62 =      41  r 59 → X
        41 ÷ 62 =       0  r 41 → F
```

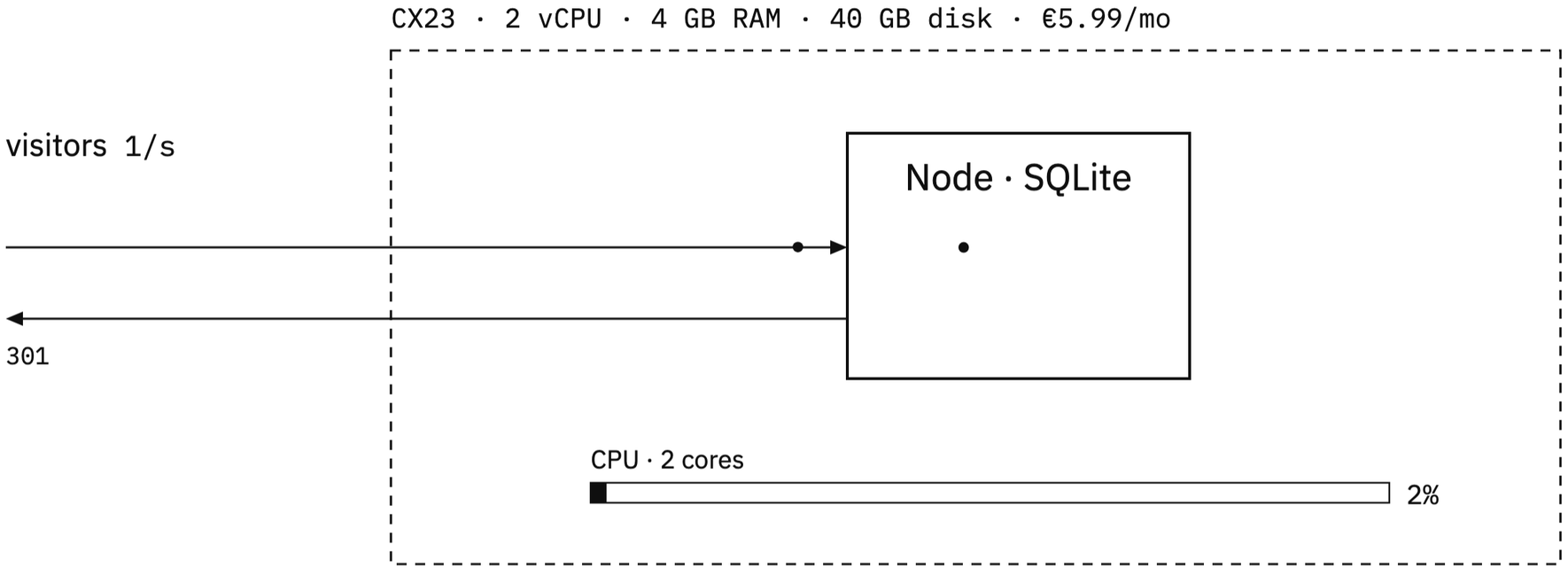
read upwards: FXsk · check: $41 \cdot 62^3 + 59 \cdot 62^2 + 28 \cdot 62 + 20 = 10{,}000{,}000$

alphabet: 0–9 a–z A–Z (62 characters) · 7 characters ≈ 3.5 trillion codes · example rows from the seed

E01 Launch, v0 ✓

1 click/s · 1 new link/min · 1,000 links

+ the always-true rules



redirect p99

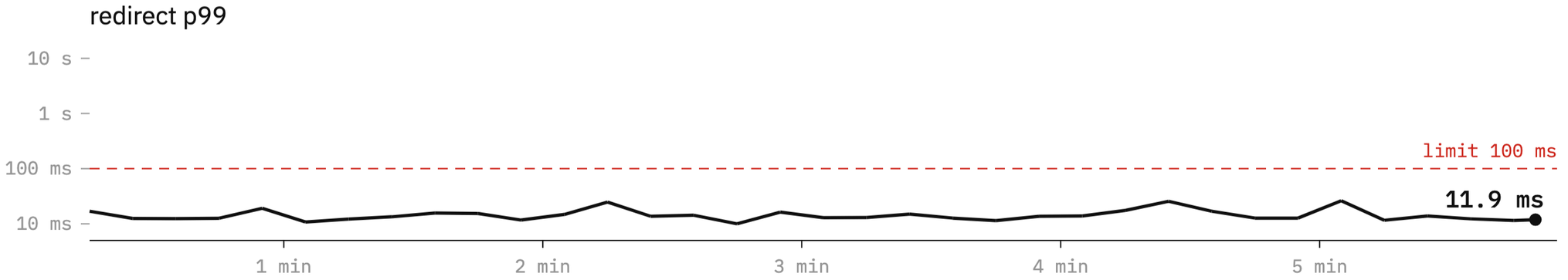
17.8 ms ✓

limit 100 ms

create p99

27.8 ms ✓

limit 300 ms



1 dot = 1 request

run e01-01

E02

Load

1% of Bitly

100 clicks a second and 2 new links a second, with 10 million links already stored.

why it matters

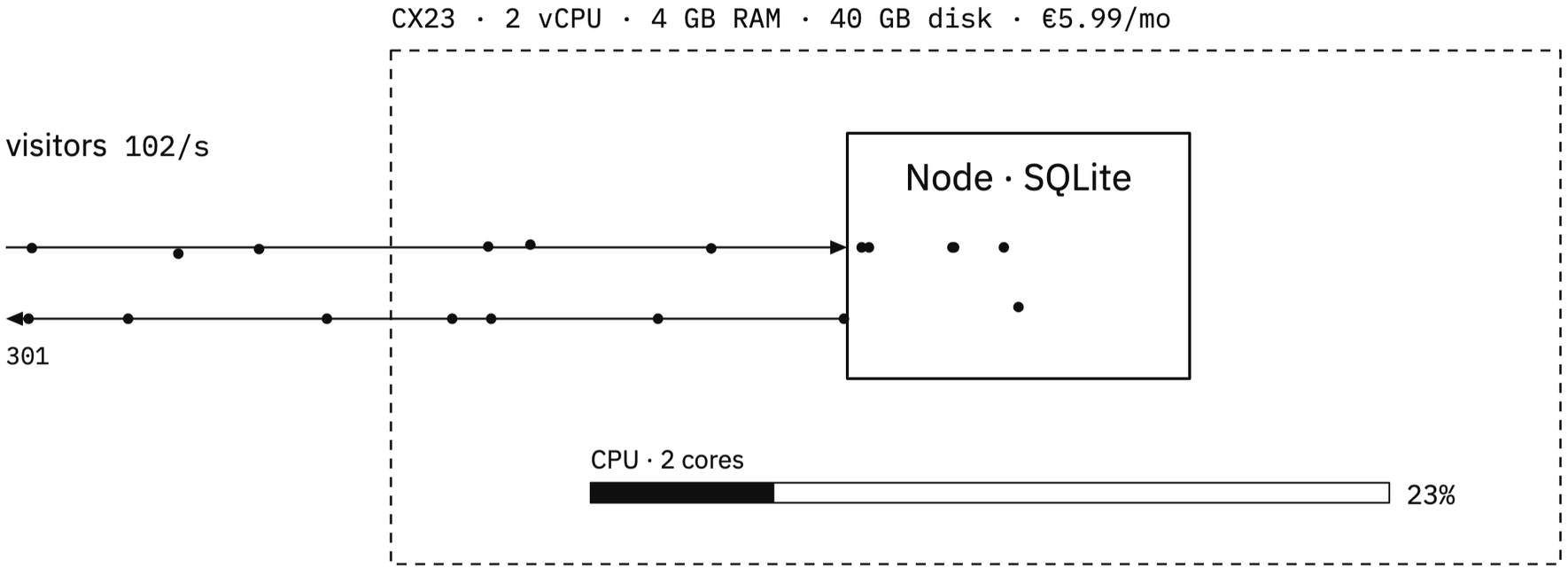
A genuinely successful product: about 88 million clicks a month. Most products never get this far.

`pass: the always-true rules`

E02 1% of Bitly, v0 ✓

100 clicks/s · 2 new links/s · 10 M links

+ the always-true rules



redirect p99

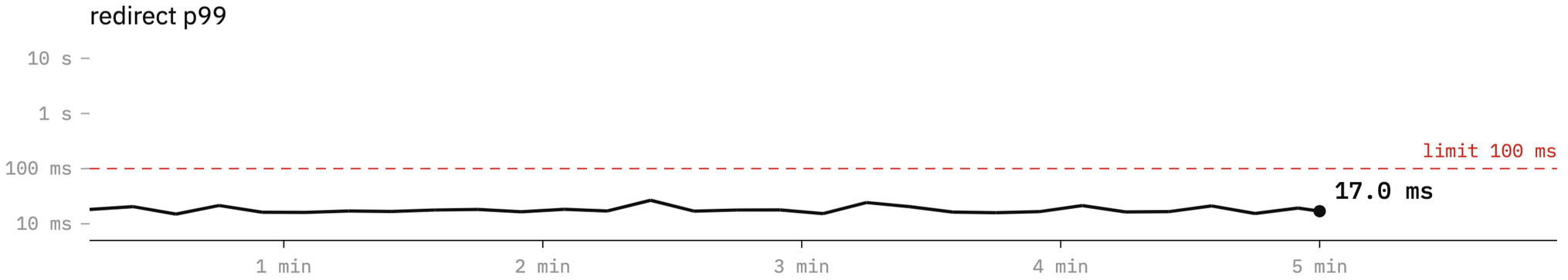
17.7 ms ✓

limit 100 ms

create p99

26.0 ms ✓

limit 300 ms



1 dot = 10 requests

run e02-01

E03

Requirement

Custom names

Creators can pick their own name. 50 people claim the same name at the same instant, 3 times a second, 1,000 rounds in all, on top of E02 traffic.

why it matters

People want readable links. The hard part is fairness: two people asking for the same name at the same moment.

pass: `exactly one winner every time; the rest told "taken"`

E03 50 people, one name, v0 ✗

50 people send this at the same instant:

```
POST /links
{ "url": "https://example.com/launch",
  "name": "launch-day" }
```

× 50

v0 only reads "url". It never looks at "name":

it was built before names existed, and unknown fields are silently ignored.

every one of the 50 gets:

```
201 saved · FXDY
201 saved · FXDZ
201 saved · FXE0
201 saved · FXE1
```

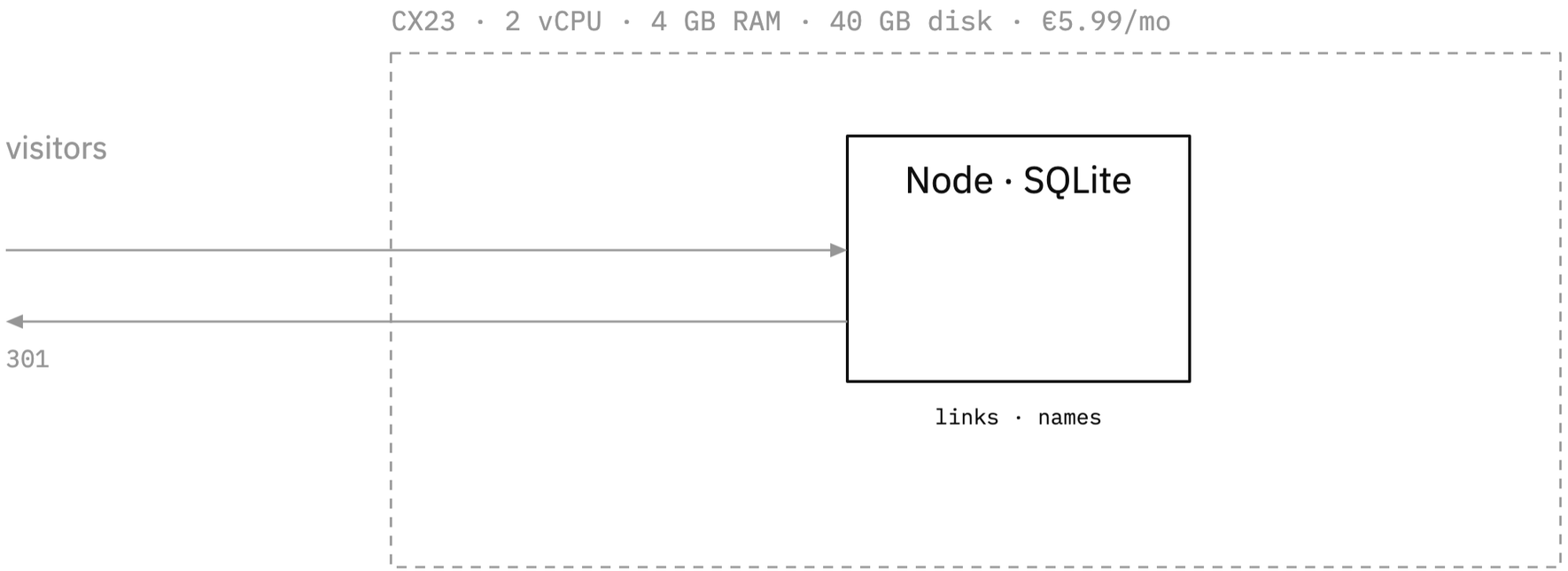
… all 50 told "saved"

and /launch-day → 404

every round, the same

run e03-01

E03 v1: a names table



Names need a hyphen or 8+ characters,

so they can never look like a generated code.

E03 v1: a second table

links

id	url	created_at
10000000	https://news.example.invalid/…/story-10000000	1791123320

names

name	url	created_at
launch-day	https://example.com/launch	1791126920

a name needs a hyphen or 8+ characters, so it never looks like a number in base 62 · example rows

After every change, rerun every earlier event

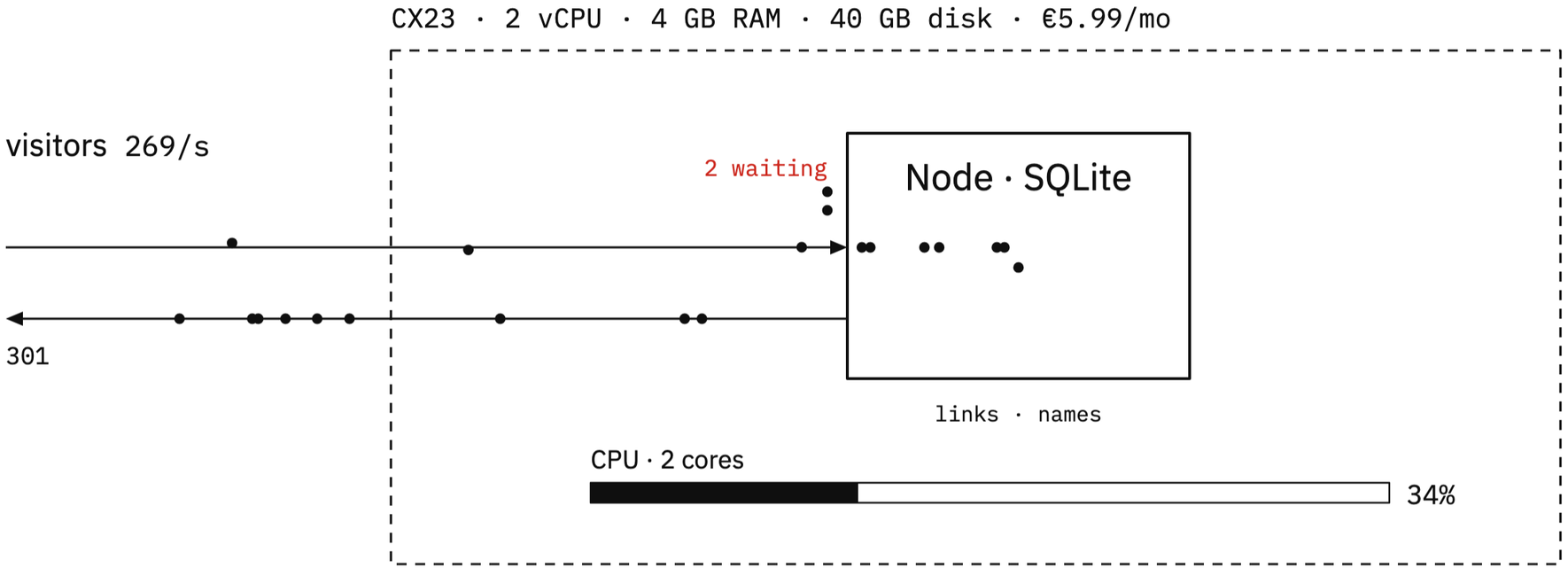
version	v0	**v1**	v2	v3	v4	v5	v6	v7	v8
E01	✓	✓							
E02	✓	✓							
E03	✗	?							
E04									
E05									
E06									
E07									
E08									
E09									

A fix that breaks an earlier event isn’t a fix.

E03 Custom names, v1 ✕

E02 traffic + 50 people claiming one name, 3 times a second

exactly one winner, the rest told "taken"



redirect p99

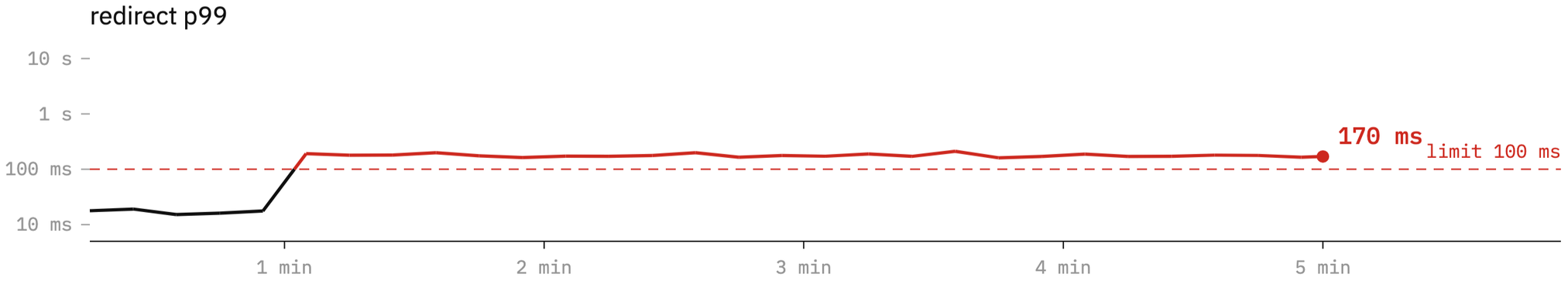
179.5 ms ✕

limit 100 ms

names

one winner, every round ✓

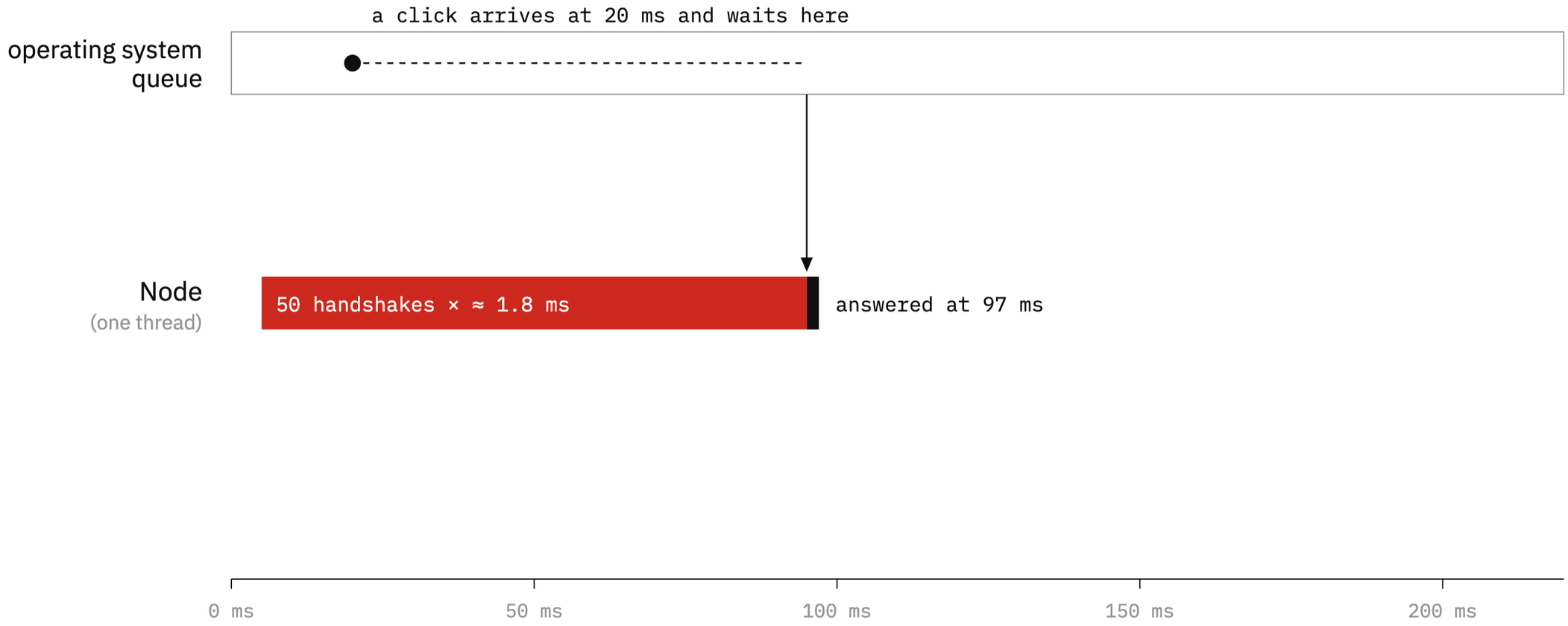
1,000 rounds



1 dot = 30 requests

run e03-02

E03 Node’s one thread, during one burst



About 3 bursts a second. Averaged over a second the CPU looks fine (34%): the wait lives inside these 90 ms.

illustration: 1.8 ms per handshake measured (calib-01), 50 claims per burst from E03, the arrival time is an example

E03 Three ways out

Two copies of the app

Handshakes on both cores, but now two processes can both think a name is free, and two writers share one SQLite file.

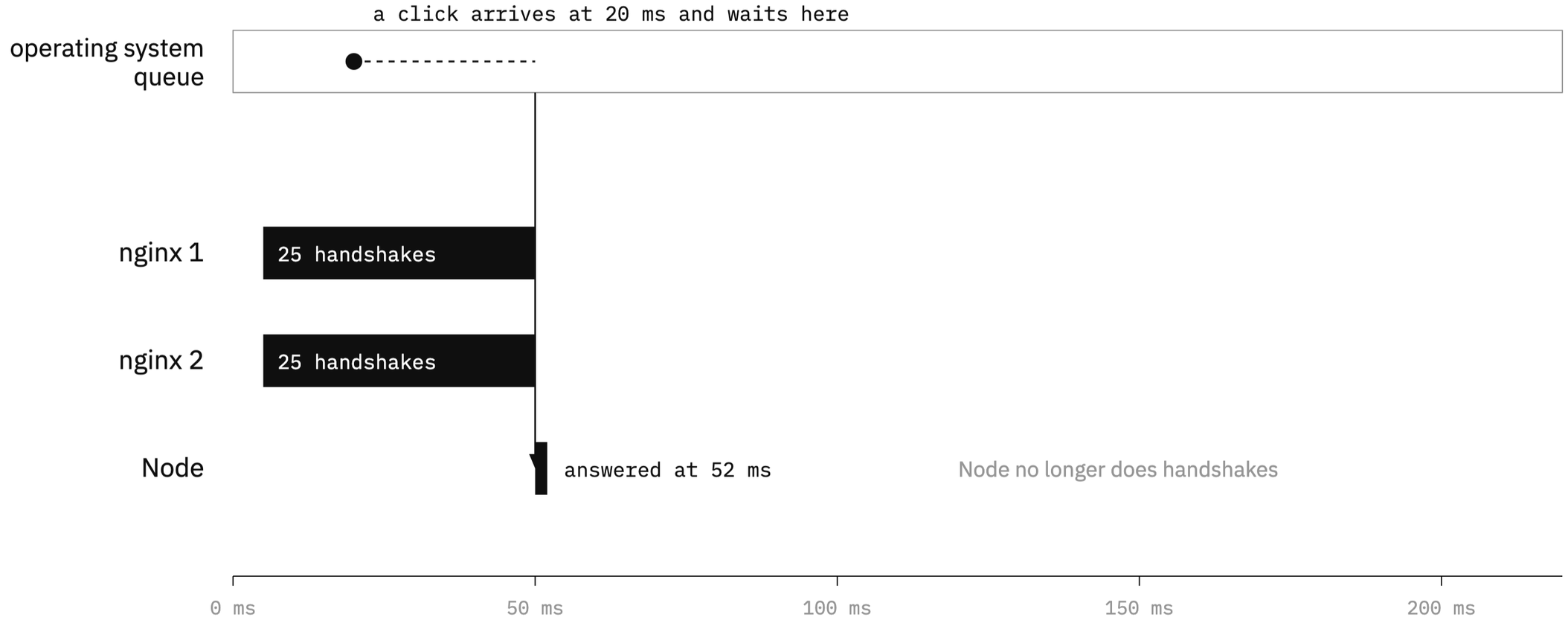
A faster server

One thread is still one thread.

✓ nginx in front, doing TLS on every core

The app stays single-threaded, so names stay race-free. No application logic changes.

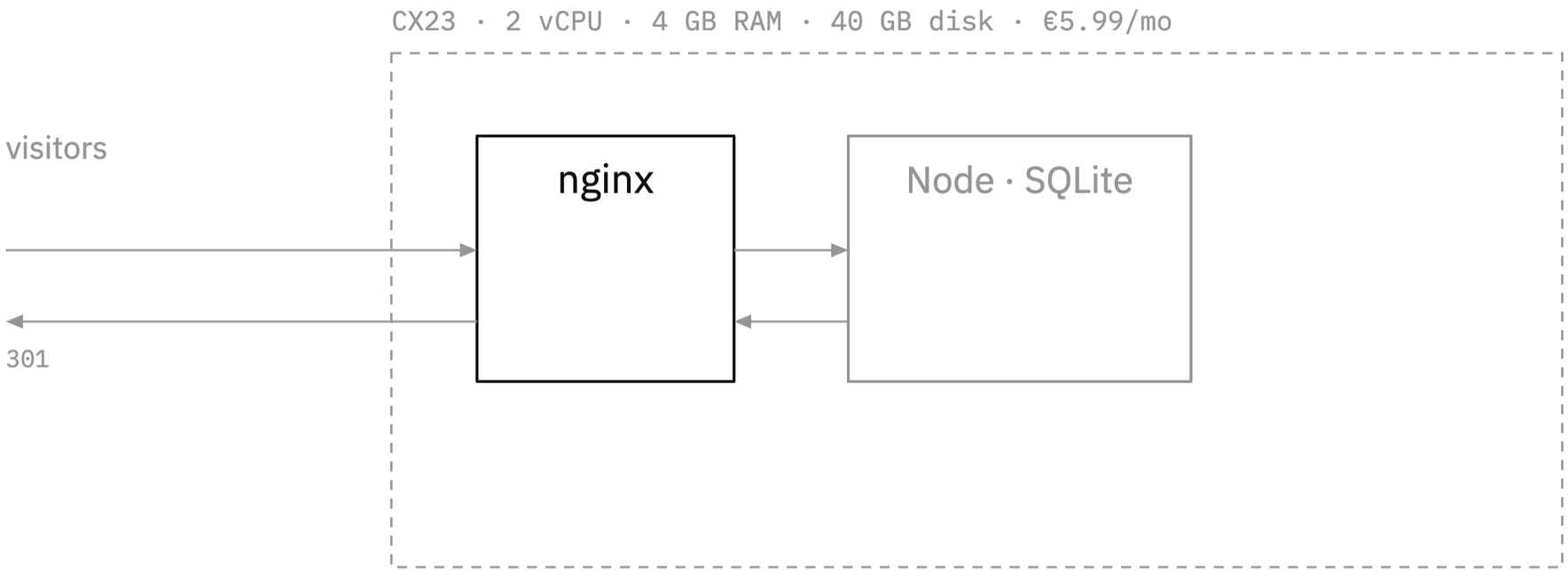
E03 With nginx: two workers share each burst



Expected before running: half the wait. Half of v1’s 180 ms p99 is about 90 ms: right at the limit, so it may or may not pass.

illustration: 1.8 ms per handshake measured (calib-01), 50 claims per burst from E03, the arrival time is an example

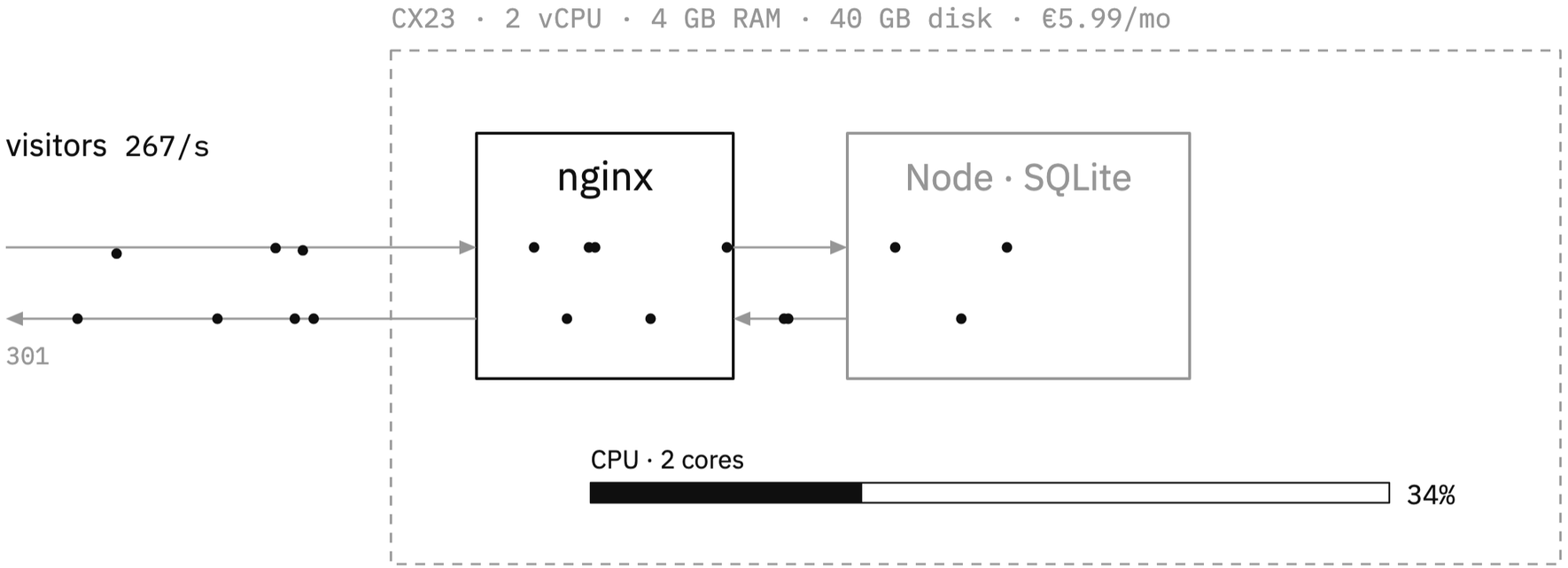
E03 v2: nginx in front



E03 Custom names, v2 ✓

E02 traffic + 50 people claiming one name, 3 times a second

exactly one winner, the rest told "taken"



redirect p99

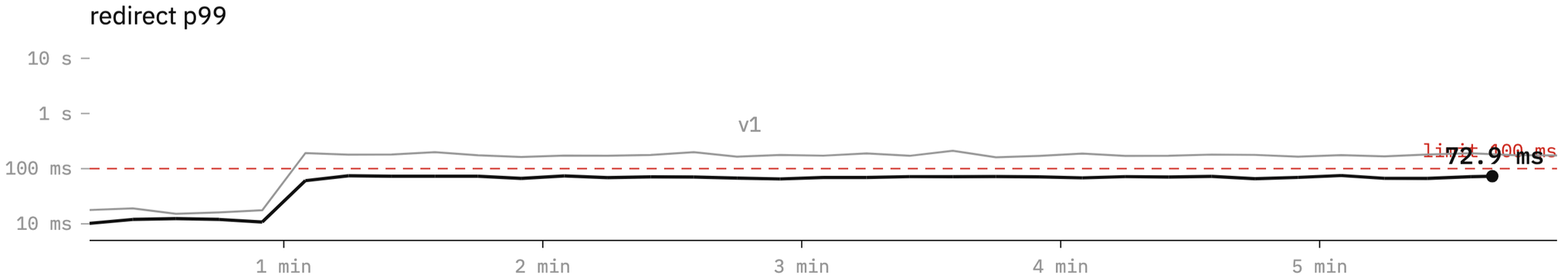
71.0 ms ✓

limit 100 ms

names

one winner, every round ✓

1,000 rounds



1 dot = 30 requests

run e03-08 · grey: e03-02

Always true, in every event

does	paste a link → a code of at most 7 letters and digits open a code → 301 to the link · unknown code → not found HTTPS only
never	lose a confirmed link, or send it to the wrong page give a code two owners
fast	`redirect p99 < 100 ms · create p99 < 300 ms · errors < 0.1%`
since	**E03** a name has exactly one owner; the rest are told "taken"

Every earlier event, rerun after every change

version	v0	v1	v2	v3	v4	v5	v6	v7	v8
E01	✓	✓	✓						
E02	✓	✓	✓						
E03	✗	✗	✓						
E04									
E05									
E06									
E07									
E08									
E09									

E04

Load

A link goes viral

On top of E02 traffic, one link ramps from 0 to 2,000 clicks a second in a minute, and stays there for 10 minutes.

why it matters

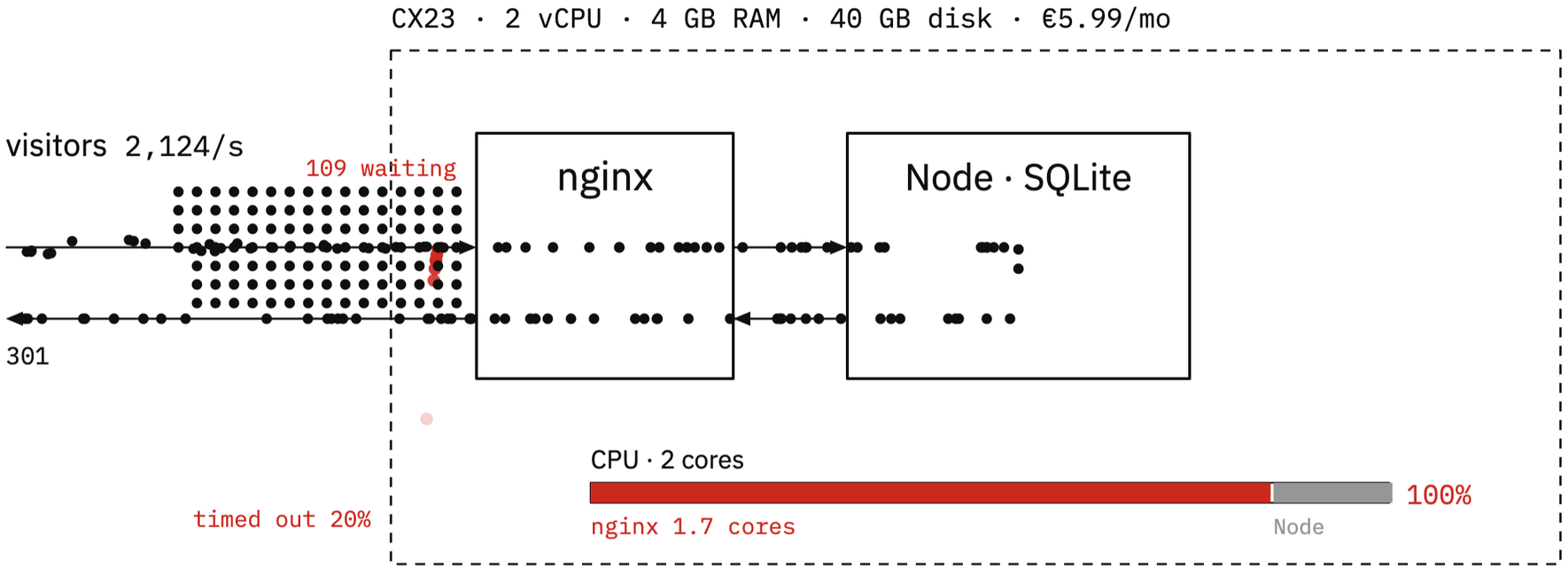
Short links get shared. One big post can send thousands of people to the same link within a minute.

pass: `the always-true rules for every link, the hot one too`

E04 2,000 clicks a second, v2 ✕

E02 traffic + one link at 2,000 clicks/s for 10 min

+ the always-true rules



redirect p99

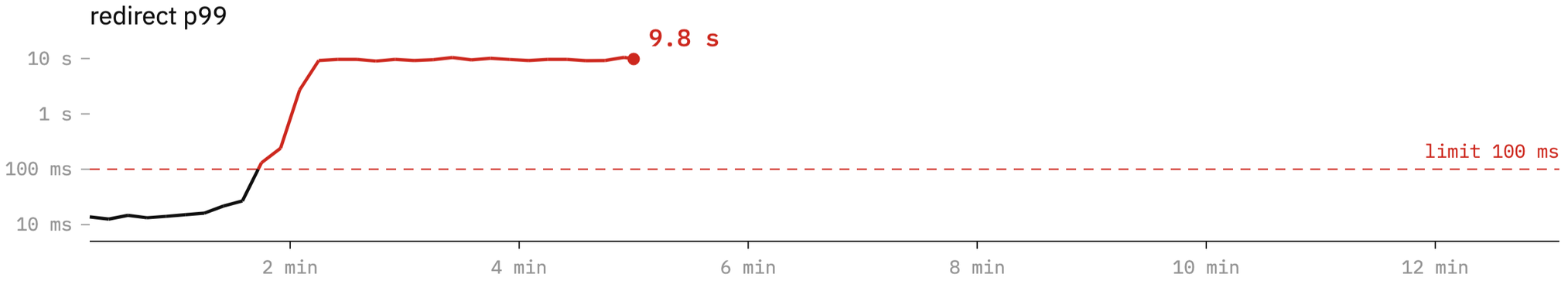
9.4 s ✕

limit 100 ms

errors

18.4% ✕

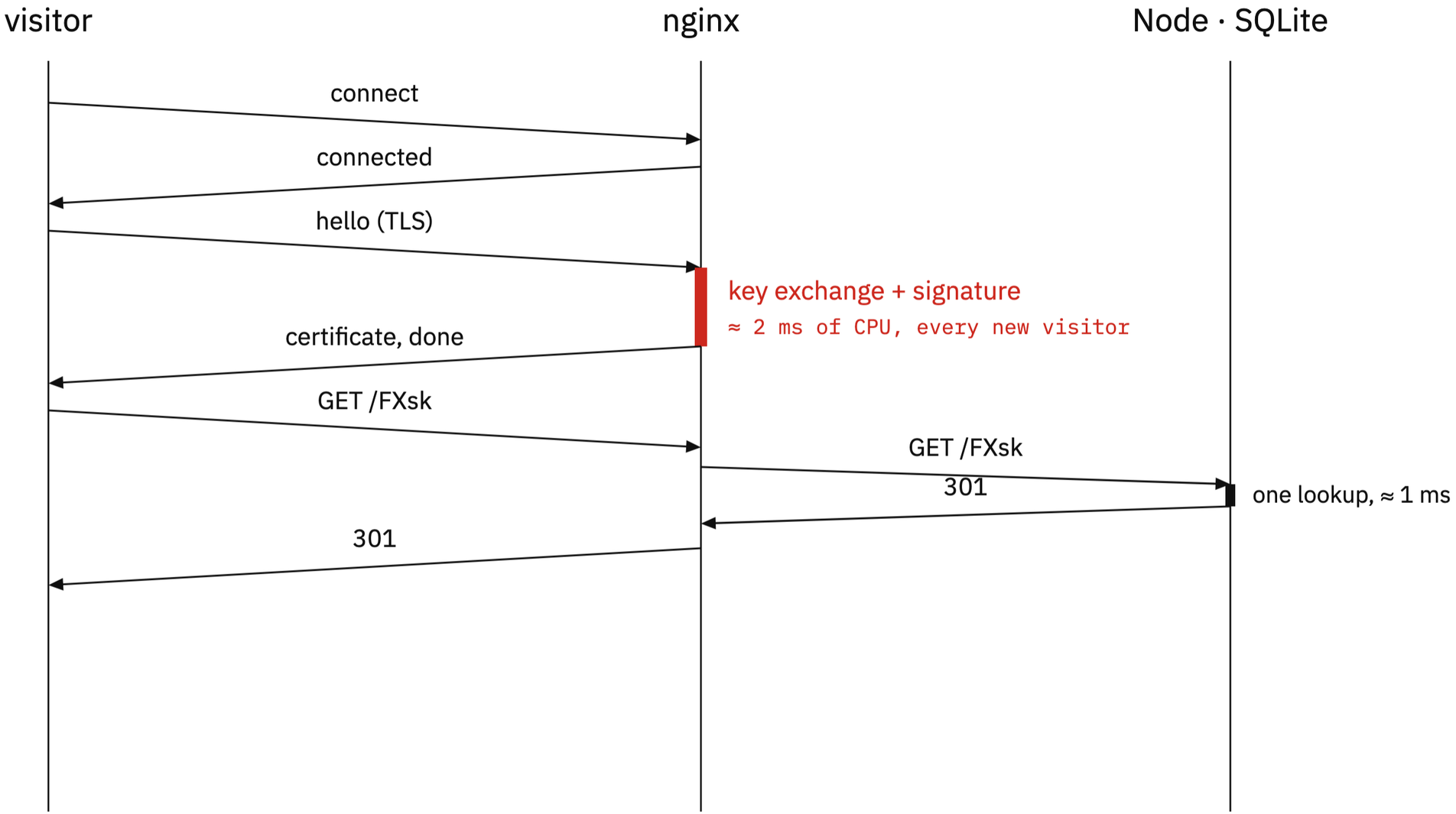
limit 0.1%



1 dot = 30 requests

run e04-01

E04 Why: one click, in time



At 2,000 new visitors a second: nginx ≈ 170% of a core, Node ≈ 30%. The machine has 200%.

run e04-01

E04 More handshakes per second: three ways

A CDN

Another company’s servers do the handshakes, close to visitors.

A load balancer

A separate machine does the handshakes: + €7.49 a month.

✓ Twice the cores: CX33

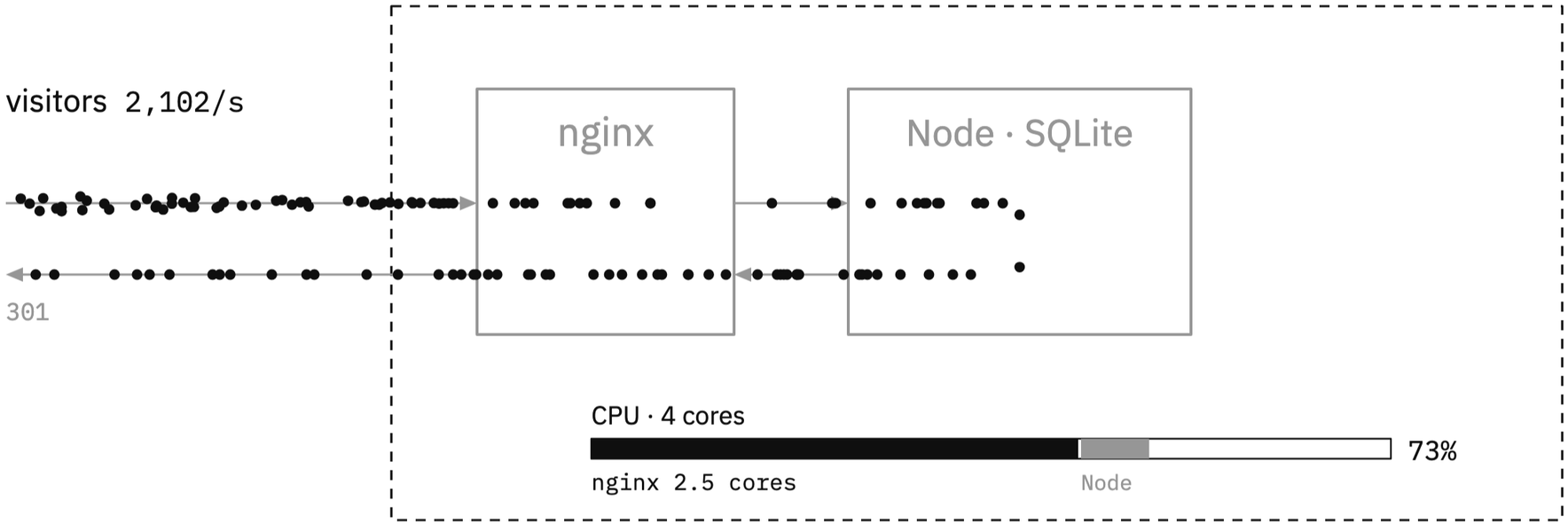
Same software, four cores instead of two: + €3 a month. The smallest change.

E04 v3: same software, 4 cores ✓

E02 traffic + one link at 2,000 clicks/s for 10 min

+ the always-true rules

~~CX23 · 2 vCPU~~ CX33 · 4 vCPU · 8 GB RAM · 80 GB disk · €8.99/mo



redirect p99

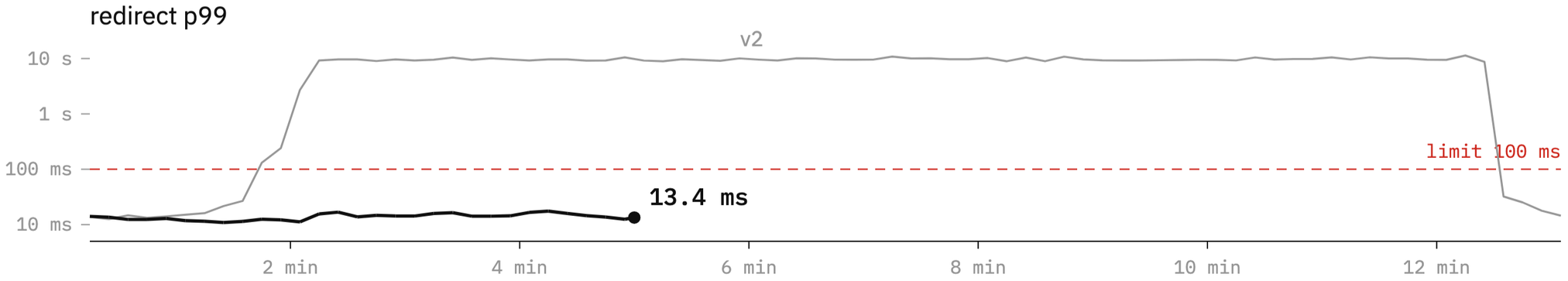
14.6 ms ✓

limit 100 ms

errors

0% ✓

limit 0.1%



1 dot = 30 requests

run e04-02 · grey: e04-01

Every earlier event, rerun after every change

version	v0	v1	v2	**v3**	v4	v5	v6	v7	v8
E01	✓	✓	✓	✓					
E02	✓	✓	✓	✓					
E03	✗	✗	✓	✓					
E04			✗	✓					
E05									
E06									
E07									
E08									
E09									

The scoreboard

version	design	passed	p99	€/month
v0	One Node program, HTTPS direct, on a CX23	E02	17.7 ms	5.99
v1	+ a names table	E02	12.9 ms	5.99
v2	+ nginx in front for TLS	E03	69.2 ms	5.99
v3	same software, 4 vCPU (CX33)	E04	14.6 ms	8.99

E05

Requirement

Click counts

Creators see clicks per link per day, at most a minute behind. Tested by rerunning the viral spike with counting on.

why it matters

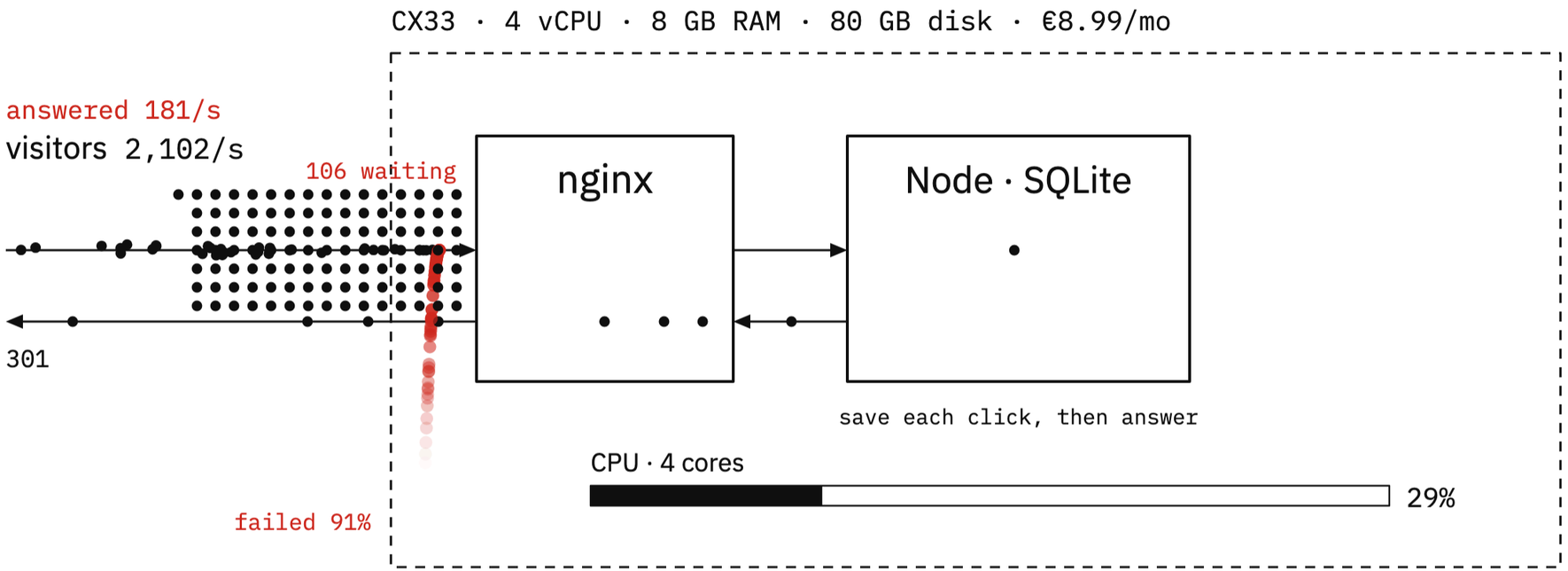
Knowing who clicked is why many people use a shortener at all. And counting must not slow the redirect down.

pass: `every link’s count within 1% of the truth, at most 60 s behind`

E05 The obvious way: save every click ✕

the E04 spike, counting every click

counts within 1%, at most 60 s behind



redirect p99

10 s ✕

limit 100 ms

errors

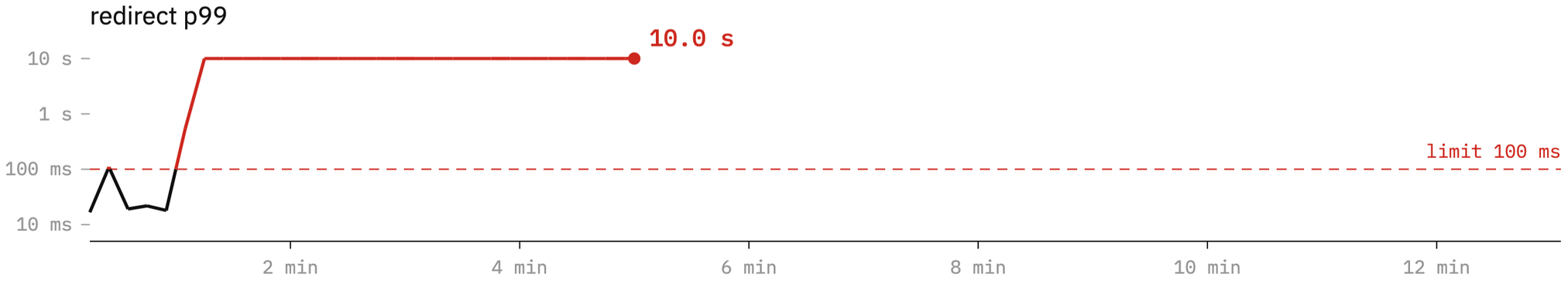
91% ✕

limit 0.1%

click counts

231 links off ✕

by more than 1% · counted 125,550, real 118,913



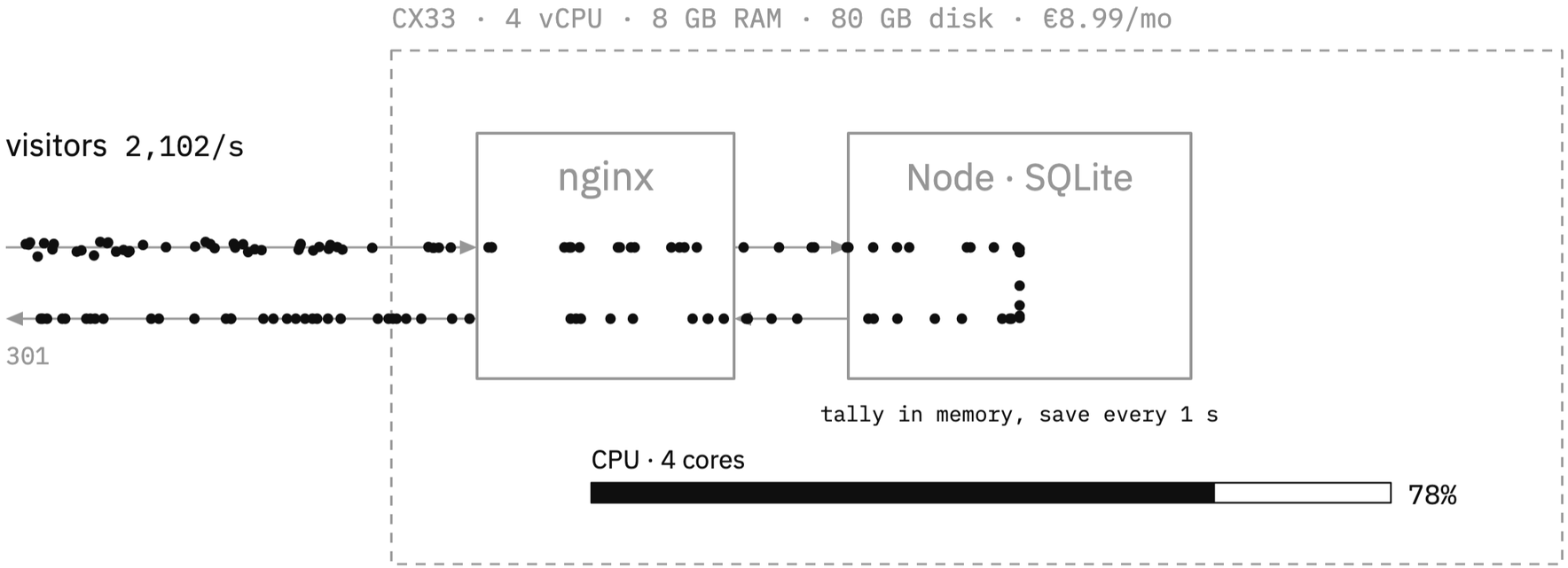
1 dot = 30 requests

run e05-01

E05 v4: count in memory, save every second ✓

the E04 spike, counting every click

counts within 1%, at most 60 s behind



redirect p99

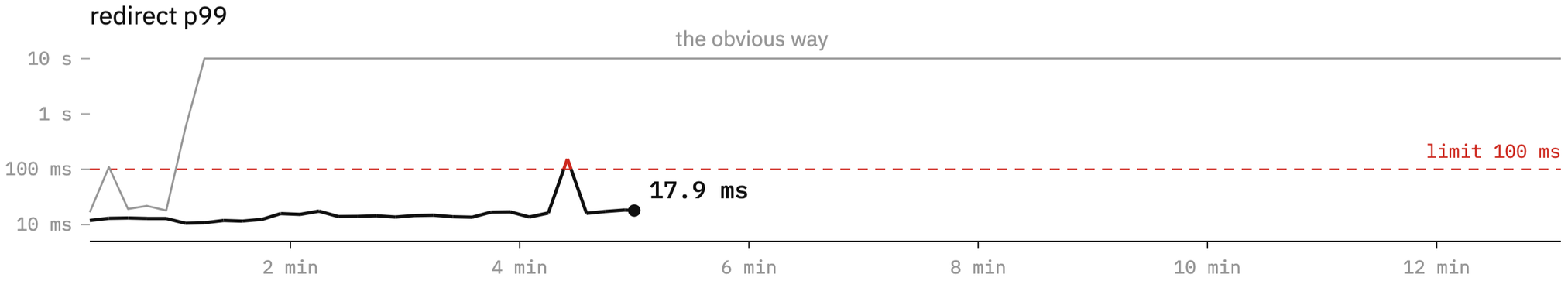
15.4 ms ✓

limit 100 ms

click counts

exact ✓

1,101 links checked



1 dot = 30 requests

run e05-03 · grey: e05-01

E05 v4: counts, and browsers must ask every time

links

id	url	created_at
10000000	https://news.example.invalid/…/story-10000000	1791123320

names

name	url	created_at

clicks

code	day	count
FXsk	2026-10-04	1284
launch-day	2026-10-04	310

every redirect now looks like:

HTTP/1.1 301 Moved Permanently

Location: https://example.com/…

Cache-Control: no-store

one row per link per day, added to once a second · indexed on (code, day), its primary key · example rows

Always true, in every event

does	paste a link → a code of at most 7 letters and digits open a code → 301 to the link · unknown code → not found HTTPS only
never	lose a confirmed link, or send it to the wrong page give a code two owners
fast	`redirect p99 < 100 ms · create p99 < 300 ms · errors < 0.1%`
since	`E03` a name has exactly one owner; the rest are told "taken" `E05` click counts within 1%, at most 60 s behind

Every earlier event, rerun after every change

version	v0	v1	v2	v3	**v4**	v5	v6	v7	v8
E01	✓	✓	✓	✓	✓				
E02	✓	✓	✓	✓	✓				
E03	✗	✗	✓	✓	✓				
E04			✗	✓	✓				
E05				✗	✓				
E06									
E07									
E08									
E09									

E06

Failure

Power cut

Under E02 traffic, the machine is switched off without warning, then on again.

why it matters

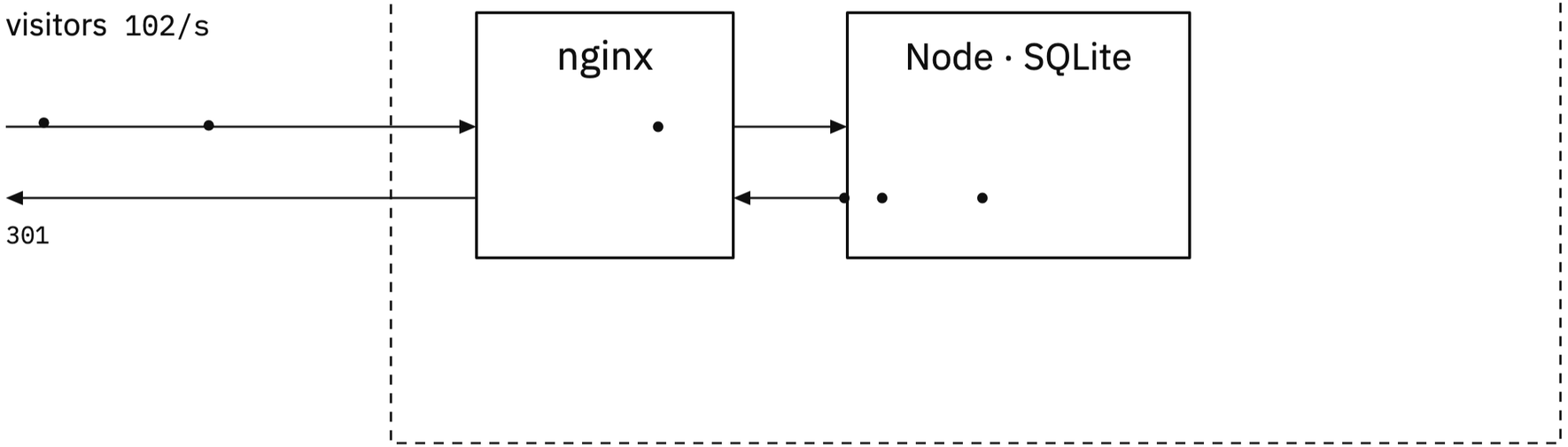
Servers crash and lose power. Whatever we told a creator was saved must still be there afterwards.

pass: `no confirmed link lost · redirects back within 5 minutes`

E06 Power cut, v4 ✕

E02 traffic, then the power is cut
back within 5 min

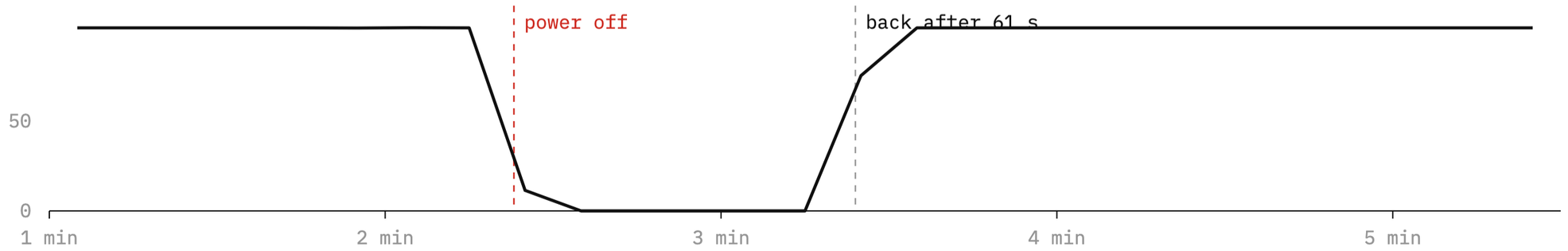
CX33 · 4 vCPU · 8 GB RAM · 80 GB disk · €8.99/mo



back after
61 s ✓
limit 5 min

confirmed links lost
1 · FXwR ✕
limit 0

redirects answered per second



1 dot = 30 requests

run e06-01

E06 Why: the save came back undone

How SQLite saves, with its default settings:

1. copy the parts of the file it will change into links.db-journal: an undo copy
2. write the new link, FXwR, into links.db
3. delete the journal: now the save counts as final
4. answer the creator: “saved, FXwR”

The catch: deleting a file is first noted only in memory. The disk records it a few seconds later.

Power cut in those seconds: after the reboot the journal is still on disk. SQLite thinks the save never finished and copies the old version back. FXwR is gone.

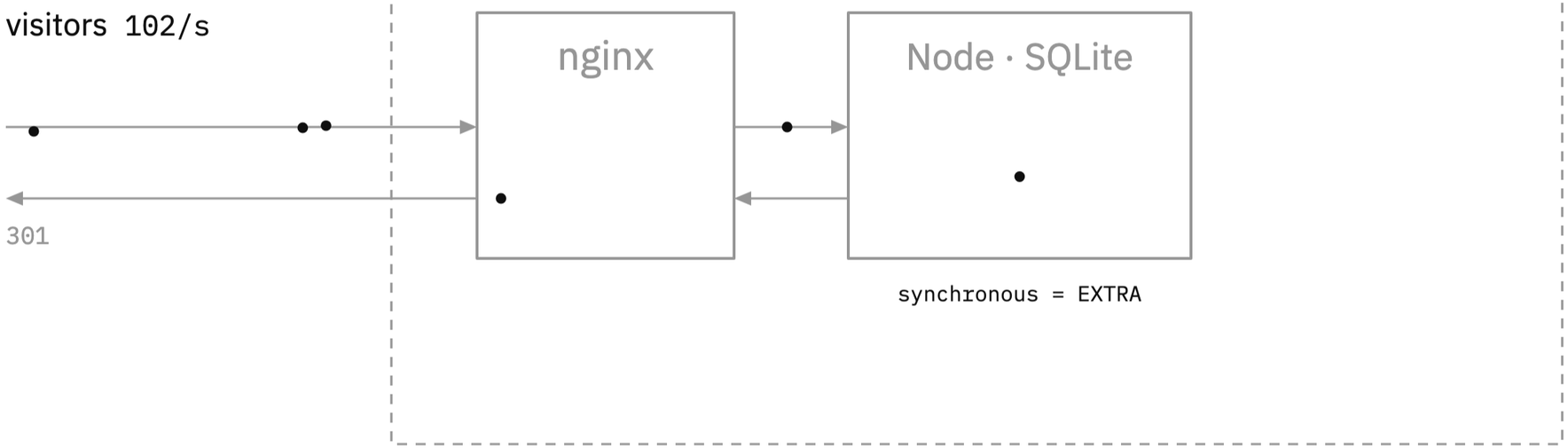
Codes count up, so the next new link gets the same number: FXwR again, now opening a stranger’s page.

```
fix · synchronous = EXTRA: answer “saved” only once the deletion is on disk
```

E06 v5: one setting ✓

E02 traffic, then the power is cut
back within 5 min

CX33 · 4 vCPU · 8 GB RAM · 80 GB disk · €8.99/mo

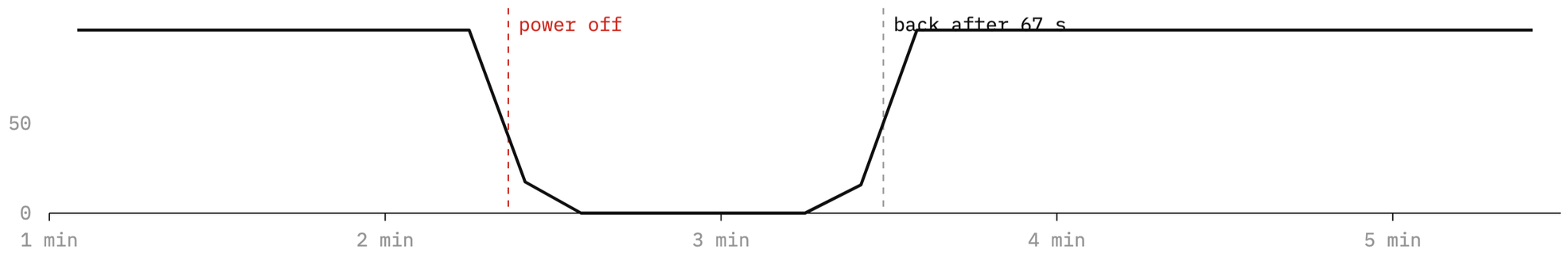


back after
67 s ✓
limit 5 min

confirmed links lost
0 of 827 ✓
limit 0

click counts
7 links short ✗
each 1 click short, all under 100 clicks, where 1% rounds down to 0 · accepted

redirects answered per second



1 dot = 30 requests

run e06-02

Every earlier event, rerun after every change

version	v0	v1	v2	v3	v4	**v5**	v6	v7	v8
E01	✓	✓	✓	✓	✓	✓			
E02	✓	✓	✓	✓	✓	✓			
E03	✗	✗	✓	✓	✓	✓			
E04			✗	✓	✓	✓			
E05				✗	✓	✓			
E06					✗	✓*			
E07									
E08									
E09									

* passes, except about 1 s of click counts lost in a crash: accepted

E07

Failure

The machine is lost

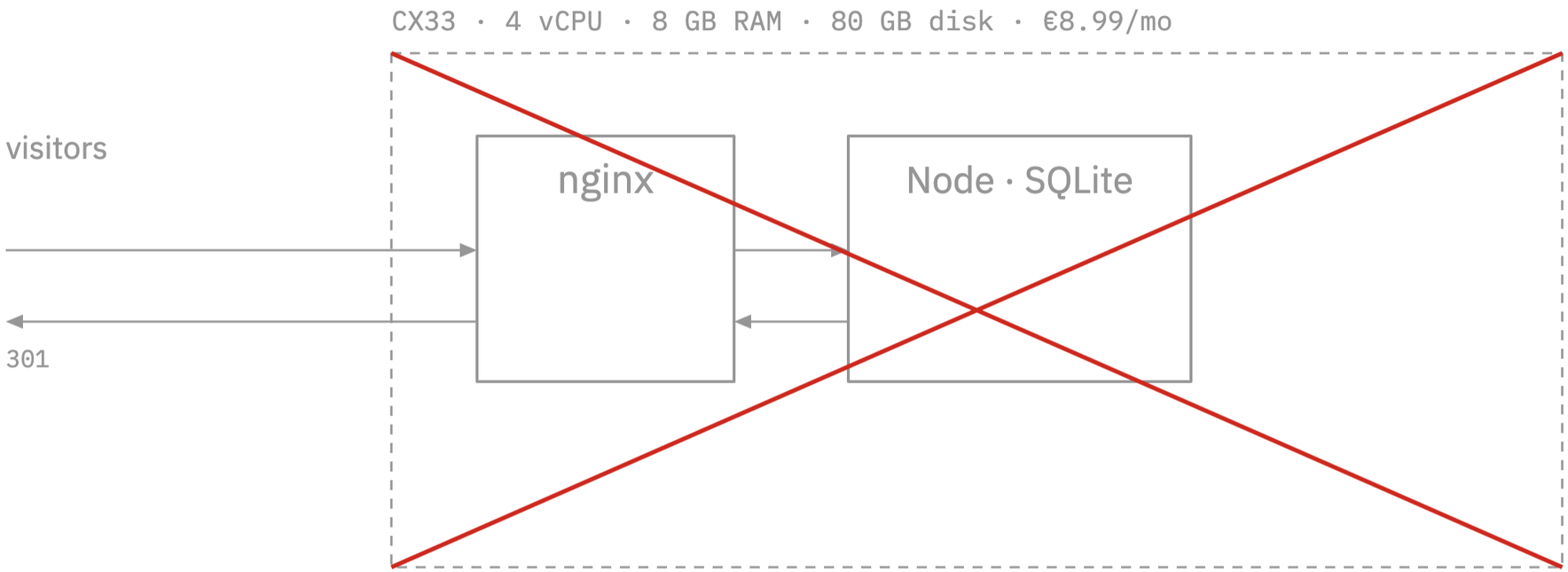
Under E02 traffic, the server and its disk are deleted for good.

why it matters

Cloud machines and disks do get lost for good. How fast can we be back, and what do we lose?

pass: `back on a new machine within 1 hour · lose at most the last 5 minutes of links`

E07 v5 can’t survive this



	v5 is missing	v6 adds
copy	the only copy of the data is on this disk	Litestream copies every change to a Volume
address	links contain this server’s IP, deleted with it	keep the IP address when the server goes
notice	nobody outside is watching	Better Stack checks /health every 30 s
rebuild	a new server would be set up by hand	one script: new server, same IP, restore
codes	after a restore, the newest codes could repeat	confirm a new link only once it’s in the copy

E07 Copy: three ways

Hetzner’s daily backups

A copy once a day, but we may lose at most 5 minutes. And backups belong to the server: deleting the server deletes them too.

A standby server

A second server, always running with a live copy, takes over in seconds. But we only have to be back within an hour, and it would double the bill every month.

✓ Litestream, copying every change to a Volume

About a second behind. A Volume is a separate disk that survives the server being deleted: €0.57 a month.

E07 Machine lost, v6 ✓

E02 traffic, then the machine is deleted
back within 1 h · may lose the last 5 min of links

CX33 · 4 vCPU · 8 GB RAM · 80 GB disk · kept IP · €8.99/mo

Better Stack checks /health every 30 s

visitors 102/s

301

nginx

Node · SQLite

Litestream

copy ≈ 1 s

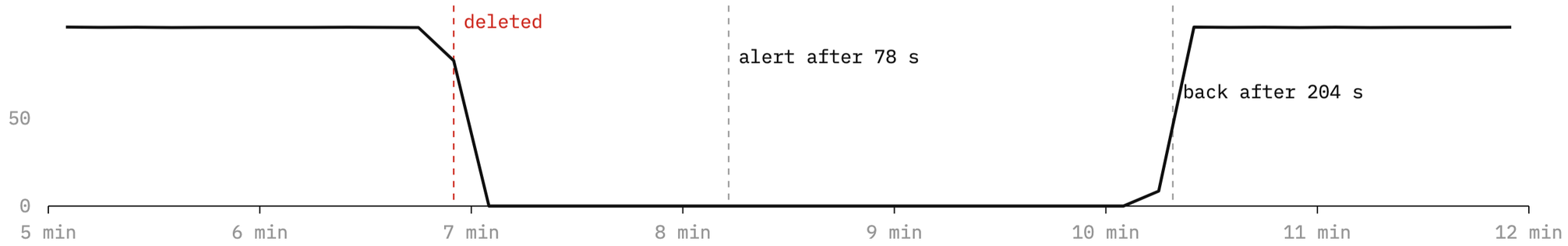
Volume
10 GB · €0.57/mo

new machine · same IP · restored from the Volume

back after
204 s ✓
limit 1 h

confirmed links lost
0 ✓
may lose the last 5 min

redirects answered per second



1 dot = 30 requests

run e07-02

Every earlier event, rerun after every change

version	v0	v1	v2	v3	v4	v5	**v6**	v7	v8
E01	✓	✓	✓	✓	✓	✓	✓		
E02	✓	✓	✓	✓	✓	✓	✓		
E03	✗	✗	✓	✓	✓	✓	✓		
E04			✗	✓	✓	✓	✓		
E05				✗	✓	✓	✓		
E06					✗	✓*	✓*		
E07						✗	✓*		
E08									
E09									

* passes, except about 1 s of click counts lost in a crash: accepted

The scoreboard

version	design	passed	p99	€/month
v0	One Node program, HTTPS direct, on a CX23	E02	17.7 ms	5.99
v1	+ a names table	E02	12.9 ms	5.99
v2	+ nginx in front for TLS	E03	69.2 ms	5.99
v3	same software, 4 vCPU (CX33)	E04	14.6 ms	8.99
v4	+ click counts in memory, saved every 1 s	E05	14.5 ms	8.99
v5	+ durable saves (synchronous = EXTRA)	E06	14.2 ms	8.99
v6	+ Litestream copy to a Volume, monitor, rebuild	E07	8.3 ms	9.56

E08

Load

10% of Bitly

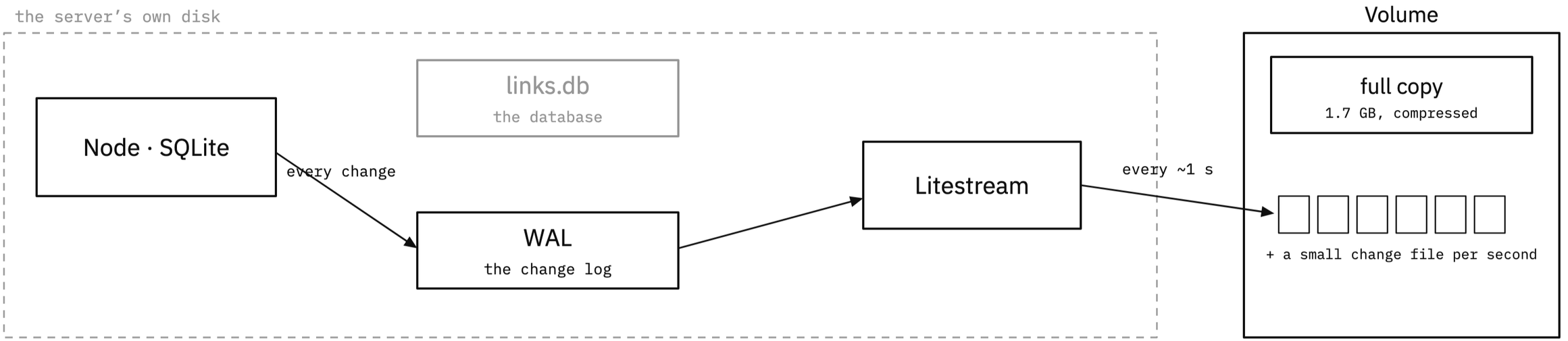
1,000 clicks a second and 20 new links a second, with 100 million links stored: 12 GB, more than the server’s memory.

why it matters

Production scale: about 877 million clicks a month. And for the first time, the data no longer fits in memory.

pass: `the always-true rules`

E08 How Litestream keeps a copy



SQLite writes every change to a change log (the WAL); about once a second, Litestream copies the new part to the Volume

Litestream’s copy = one full copy of the database + a small change file for every second since

to restore: take the full copy, then replay the small changes on top

a fresh full copy is made when Litestream starts on a new database (after loading data, or after a restore), and once a day

1.7 GB: our 12 GB database at 100 M links, compressed

since v6, every new link waits for its own small change file before we say “saved”

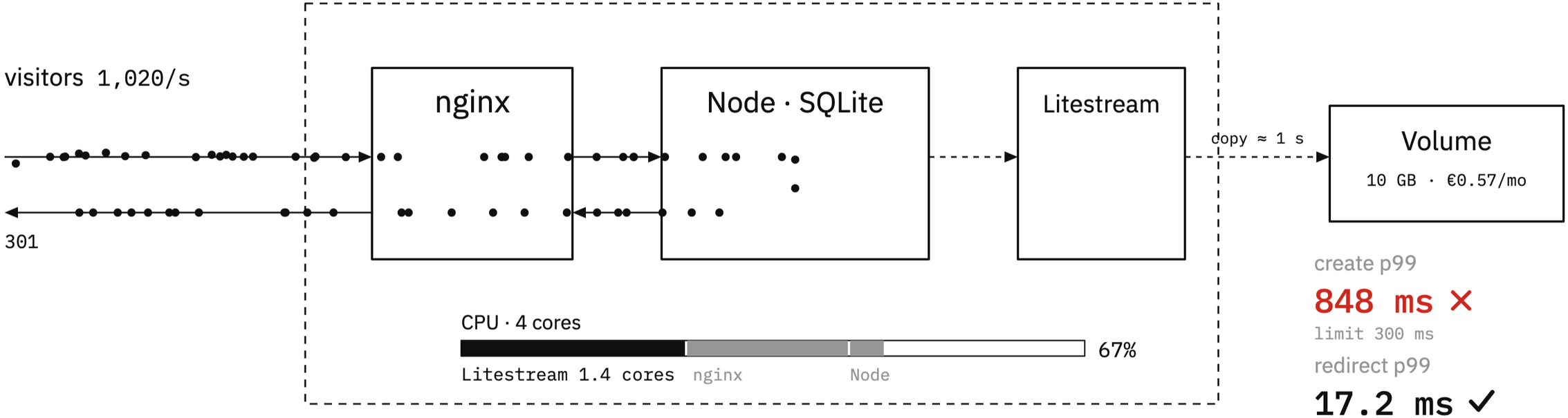
E08 10% of Bitly, v6 ✗

1,000 clicks/s · 20 new links/s · 100 M links (12 GB)

+ the always-true rules

CX33 · 4 vCPU · 8 GB RAM · 80 GB disk · kept IP · €8.99/mo

Better Stack checks /health every 30 s



create p99

848 ms ✗

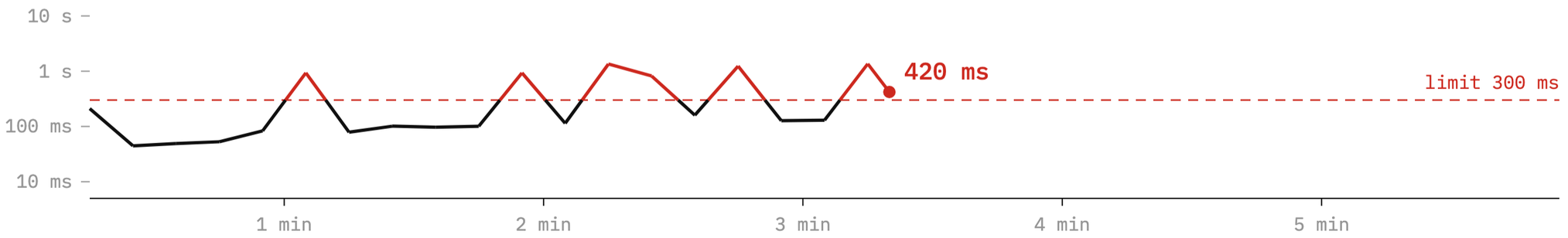
limit 300 ms

redirect p99

17.2 ms ✓

limit 100 ms

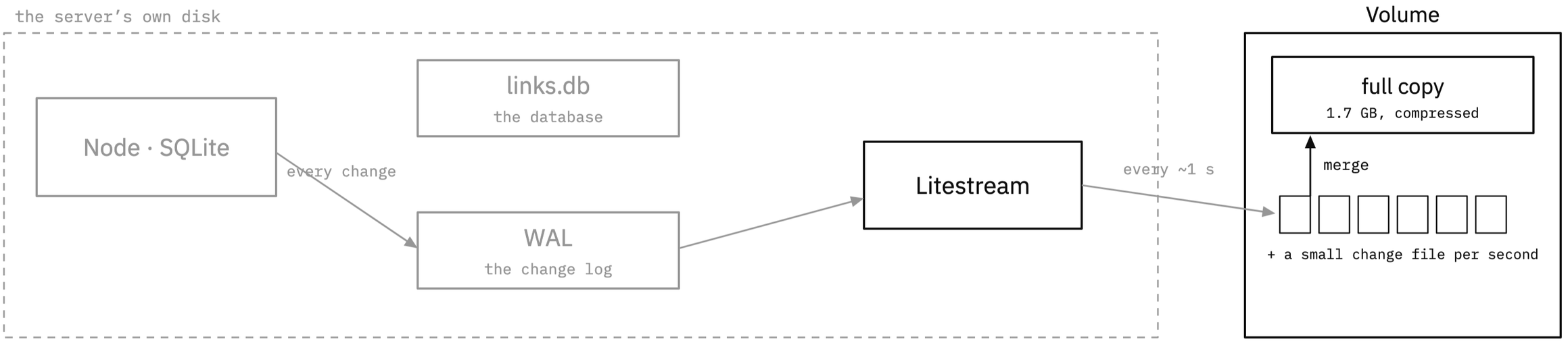
create p99



1 dot = 30 requests

run e08-02

E08 Why Litestream tidies its copy



one small file a second is 3,600 an hour, and restoring would replay them all

so Litestream merges them into bigger files: every few minutes, then hourly

a merge reads a group of files and writes them out as one new file

after a fresh full copy, the next merges include the full copy itself, so they read and rewrite all 1.7 GB

while that runs, each new link's small file waits in line behind it: 1–1.5 s

E08 Slow creates: what to change

A bigger server

The CPU and disk weren’t full. The copies queue one at a time.

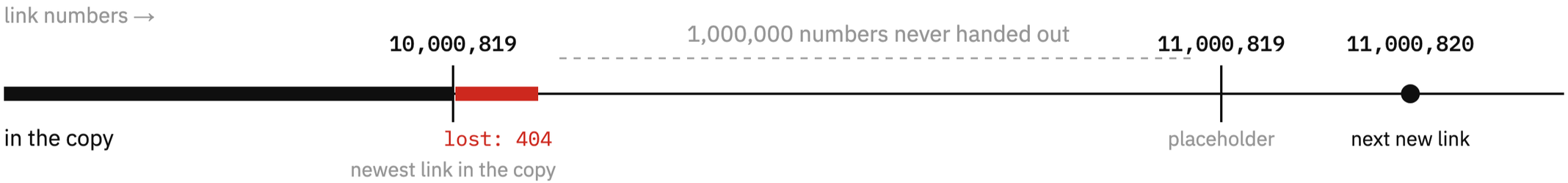
Tune Litestream to rewrite less often

Can’t remove the rewrite after a fresh copy or the daily one.

✓ Confirm links on local disk; jump the counter after a restore

A link confirmed in the machine’s last second can be lost (404), but its code is never reused. Names still wait for the copy.

E08 v7: the counter jump



after a restore, before the app starts: a placeholder goes in a million numbers ahead

links confirmed in the old machine’s last second are lost (404, which E07 allows), but their codes are never handed out again

why a million: at E08’s 20 new links a second, it covers about 14 hours, and the copy is only ever about a second behind

names can’t jump: a lost name could be registered by someone else, so names still wait for the copy

names are a small share of creates: E08’s 20 a second were all generated links

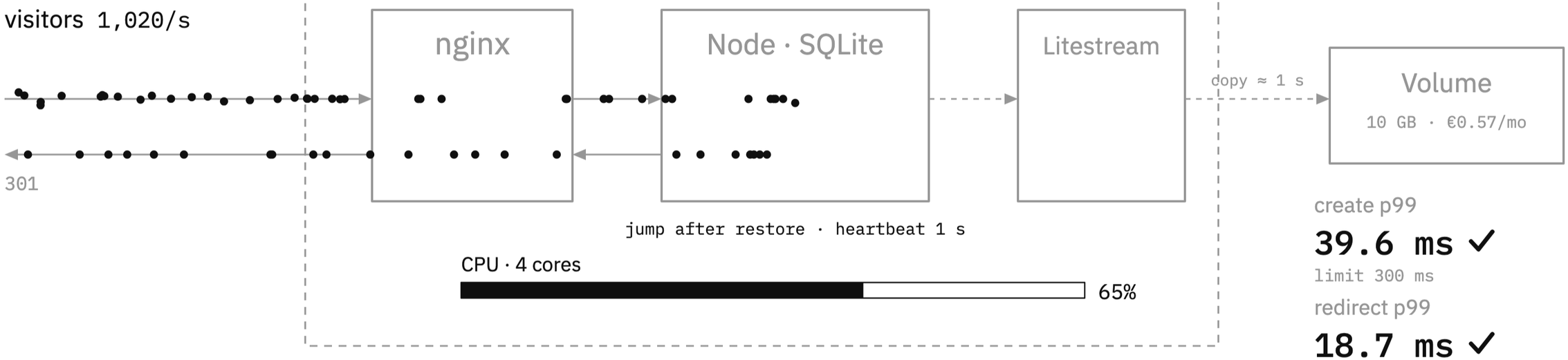
E08 v7: counter jump and heartbeat ✓

1,000 clicks/s · 20 new links/s · 100 M links (12 GB)

+ the always-true rules

CX33 · 4 vCPU · 8 GB RAM · 80 GB disk · kept IP · €8.99/mo

Better Stack checks /health every 30 s



create p99

39.6 ms ✓

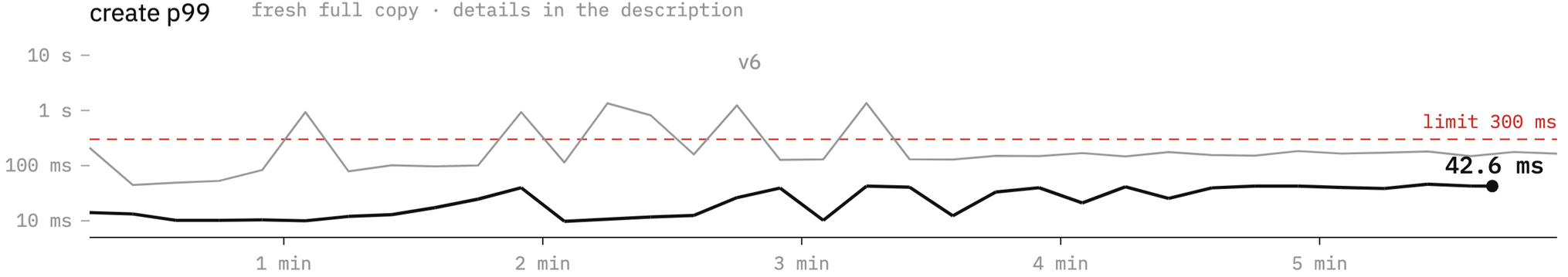
limit 300 ms

redirect p99

18.7 ms ✓

limit 100 ms

v7 also saves a one-row heartbeat every second: it avoids a rare 30-second freeze right after a fresh full copy · details in the description



1 dot = 30 requests

run e08-07 · grey: e08-02

Every earlier event, rerun after every change

version	v0	v1	v2	v3	v4	v5	v6	**v7**	v8
E01	✓	✓	✓	✓	✓	✓	✓	✓	
E02	✓	✓	✓	✓	✓	✓	✓	✓	
E03	✗	✗	✓	✓	✓	✓	✓	✓	
E04			✗	✓	✓	✓	✓	✓	
E05				✗	✓	✓	✓	✓	
E06					✗	✓*	✓*	✓*	
E07						✗	✓*	✓*	
E08							✗	✓	
E09									

* passes, except about 1 s of click counts lost in a crash: accepted

E09

Requirement

Takedowns

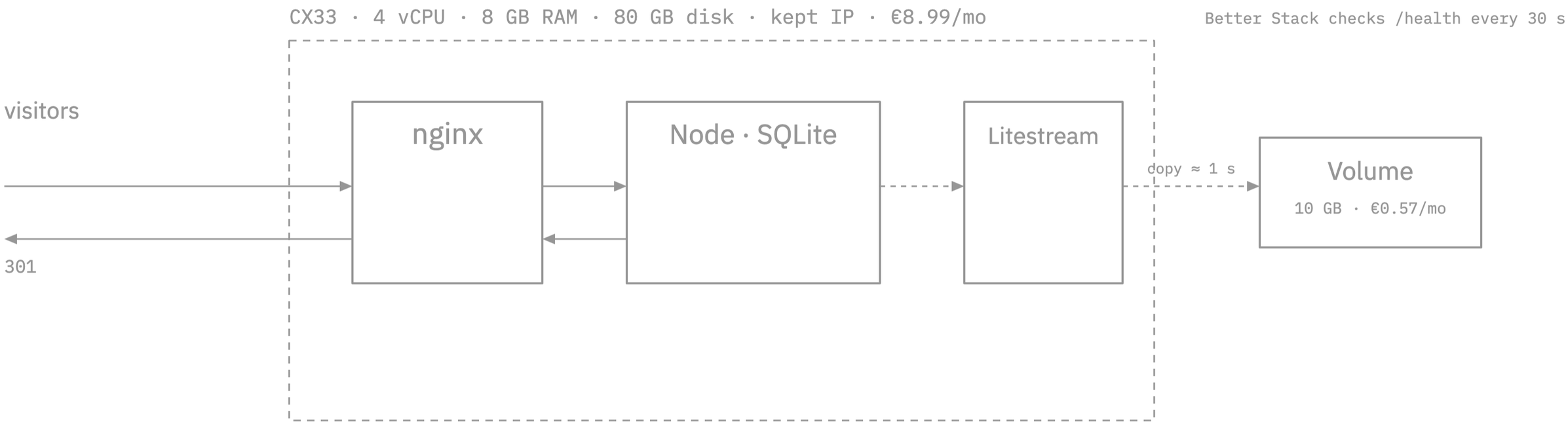
Under E08 traffic, 100 links are reported and taken down during the run.

why it matters

You can’t see where a short link goes until you click it, which makes them popular for scams. When one is reported, an operator must be able to switch it off fast.

pass: `no taken-down link redirects more than 60 s after its takedown`

E09 v7: no way to take a link down



There is no takedown at all: every reported link keeps redirecting.

· run e09-01: 0 of 101 takedowns confirmed

E09 v8: how a takedown works

links

id	url	created_at	taken_down_at
10000000	https://news.example.invalid/…	1791123320	1791552014
10000001	https://news.example.invalid/…	1791123320	

names

name	url	created_at	taken_down_at
launch-day	https://example.com/launch	1791126920	
free-prize	https://scam.example/…	1791127000	1791552100

the operator sends:

POST /links/FXsk/takedown

Authorization: Bearer <secret>

→ 200, once it’s in the copy

next click on FXsk → 410 Gone

one new column on both tables: empty = live, a time = taken down · a taken-down name stays registered, so nobody can claim it again · example rows

E09 v8: takedowns ✓

E08 traffic + 100 takedowns

no redirect later than 60 s after a takedown

CX33 · 4 vCPU · 8 GB RAM · 80 GB disk · kept IP · €8.99/mo

Better Stack checks /health every 30 s

visitors 1,020/s

301

nginx

Node · SQLite

Litestream

POST /takedown → 410 Gone

CPU · 4 cores

40%

copy ≈ 1 s

Volume

10 GB · €0.57/mo

takedowns

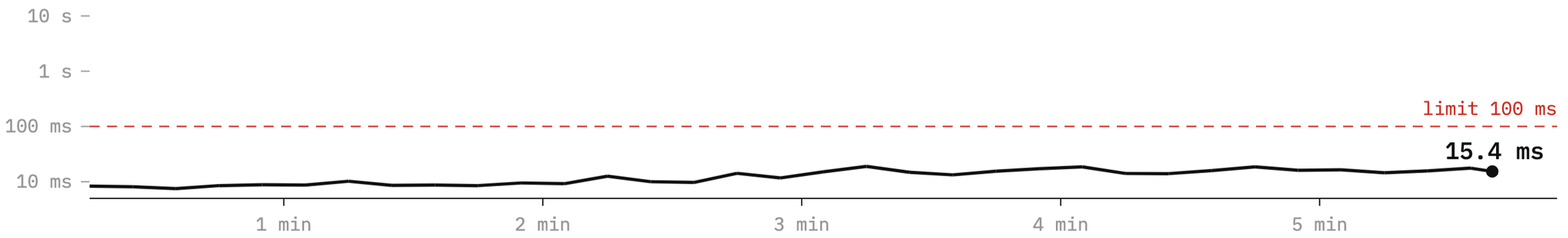
100 of 100 ✓

redirects after a takedown

0 ✓

limit: none after 60 s

redirect p99



1 dot = 30 requests

run e09-02

Always true, in every event

does	paste a link → a code of at most 7 letters and digits open a code → 301 to the link · unknown code → not found HTTPS only
never	lose a confirmed link, or send it to the wrong page give a code two owners
fast	`redirect p99 < 100 ms · create p99 < 300 ms · errors < 0.1%`
since	**E03** a name has exactly one owner; the rest are told "taken" **E05** click counts within 1%, at most 60 s behind **E09** a taken-down link stays down

Every earlier event, rerun after every change

version	v0	v1	v2	v3	v4	v5	v6	v7	**v8**
E01	✓	✓	✓	✓	✓	✓	✓	✓	✓
E02	✓	✓	✓	✓	✓	✓	✓	✓	✓
E03	✗	✗	✓	✓	✓	✓	✓	✓	✓
E04			✗	✓	✓	✓	✓	✓	✓
E05				✗	✓	✓	✓	✓	✓
E06					✗	✓*	✓*	✓*	✓*
E07						✗	✓*	✓*	✓*
E08							✗	✓	✓
E09								✗	✓

* passes, except about 1 s of click counts lost in a crash: accepted

E07 on v8 also ran on some slow Hetzner hosts (p99 100–118 ms): accepted

The scoreboard

version	design	passed	p99	€/month
v0	One Node program, HTTPS direct, on a CX23	E02	17.7 ms	5.99
v1	+ a names table	E02	12.9 ms	5.99
v2	+ nginx in front for TLS	E03	69.2 ms	5.99
v3	same software, 4 vCPU (CX33)	E04	14.6 ms	8.99
v4	+ click counts in memory, saved every 1 s	E05	14.5 ms	8.99
v5	+ durable saves (synchronous = EXTRA)	E06	14.2 ms	8.99
v6	+ Litestream copy to a Volume, monitor, rebuild	E07	8.3 ms	9.56
v7	+ counter jump after restore, heartbeat	E08	14.5 ms	9.56
v8	+ takedowns (410 Gone)	E09	14.8 ms	9.56

scoreboard.md

The final claim

1,000/s at peak = 333/s on average × 2,629,800 s a month

≈ 877 million clicks a month

20/s at peak = 6.7/s on average × 2,629,800 s

≈ 17.5 million new links a month

with 100 million links stored (12 GB, more than the server's memory)

redirect p99 14.8 ms · create p99 26.2 ms · 0 errors

€9.56 a month

p99s: median of the three E09 runs on v8

Caveats

One event at a time, and failures at a smaller scale

Combinations, like a viral spike during a power cut, were never tested. The power cut and the machine loss ran at 1% traffic with 10 million links; at 100 million, restoring alone takes about 3 minutes.

Measured next door

The load generator sat in the same data centre, so the times don’t include visitors’ own internet connection.

One machine, one data centre

The copy lives in the same location, so losing the whole data centre isn’t covered.

No domain

Links used the server’s IP address and a self-made certificate. A real service needs a domain and a proper certificate.